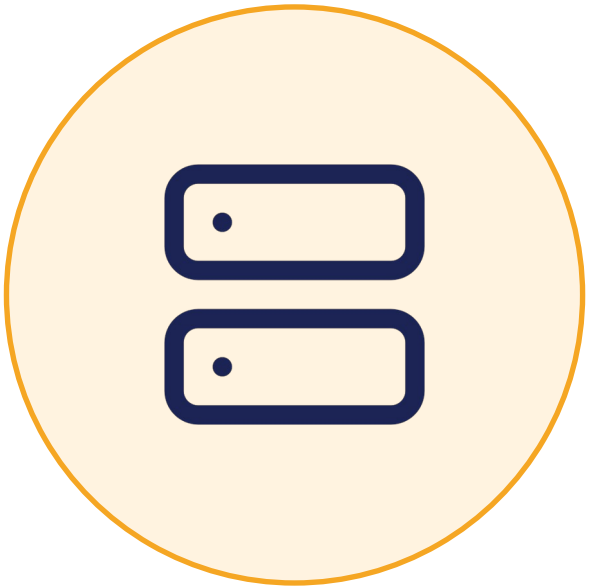
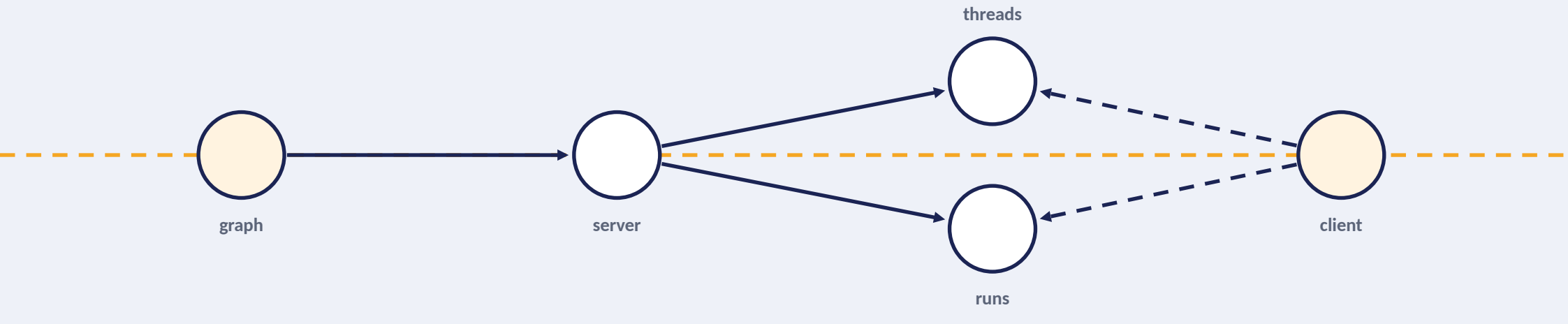
LANGGRAPH · CHAPTER 16 · FLASHCARDS

Deployment



Graphs other programs can call

12 code cards on serving a graph over HTTP and from FastAPI



Register the graph

langgraph.json names each compiled graph; langgraph dev serves it over HTTP

- The key in "graphs" is the assistant_id clients send; the path points at a compiled object.
- The dev server keeps threads in memory, with no key, database or Docker: for learning only.

BASH

```
$ cat langgraph.json
{
  "dependencies": ["."],
  "graphs": {
    "commands": "./server/graph.py:graph"
  }
}
# "commands"                -> the assistant_id every client sends
# "./server/graph.py:graph" -> the compiled object in that file:
#       graph = builder.compile()     # no checkpointer: the server stores threads
# "dependencies": ["."]     -> this project must be installed

$ python -m pip install -e .
$ langgraph dev --no-browser          # API on http://127.0.0.1:2024
$ langgraph dev --no-browser --port 2025   # port taken? then SERVER_URL=...:2025
```

Eat in or take away

A run on a thread remembers earlier runs; a run with no thread returns once and forgets

- The stream is server-sent events on the same HTTP response, one way, not a WebSocket.
- A stateless run's thread is deleted when it finishes unless on_completion="keep".

PYTHON

```python
from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://127.0.0.1:2024")

# Eat in: a table (thread) that keeps every run's state
thread_id = client.threads.create()["thread_id"]       # POST /threads
# POST /threads/{thread_id}/runs/stream
for chunk in client.runs.stream(
    thread_id,
    "commands",                                         # the assistant_id
    input={"pending_command": {"type": "PING"}},
    stream_mode="values",
):
    print(chunk.event, chunk.data)

# Take away: no thread kept, the final state in one JSON body
state = client.runs.wait(None, "commands",              # POST /runs/wait
                         input={"pending_command": {"type": "GET_STATUS"}})
# Each takeaway PING gets a fresh thread, so the count would never climb.
# Also valid: POST /runs/stream and POST /threads/{id}/runs/wait
```

A command is a state slot

pending_command is plain data in one state field, not langgraph.types.Command

- None means the slot is empty; a key a node doesn't return keeps its old value.
- total=False lets keys be absent on a fresh thread; it fills no defaults and checks no JSON.

PYTHON

```python
class PendingCommand(TypedDict):
    # The only key the router reads
    type: Literal["PING", "GET_STATUS"]

class StatusResponse(TypedDict):
    status: Literal["ok"]
    ping_count: int

class State(TypedDict, total=False):
    pending_command: PendingCommand | None   # one slot, None = empty; not a queue
    response: StatusResponse | None          # the reply the client reads
    ping_count: int                          # per thread: proves an earlier PING ran

# GET_STATUS on a thread that was never pinged, saved state afterwards:
# {'pending_command': None, 'response': {'status': 'ok', 'ping_count': 0}}
#  <- the top-level ping_count key is still absent
```

One router, a table of handlers

The router looks up the handler, calls it, and clears the slot in the same return

- Handlers are plain functions, not nodes: a new command is a new dict entry, not a new edge.
- The clear comes after the handler's keys, so a handler can't leave the command in place.

PYTHON

```python
def handle_ping(state: State) -> State:
    count = state.get("ping_count", 0) + 1
    return {"ping_count": count, "response": None}   # no stale status reply

def handle_get_status(state: State) -> State:
    return {"response": {"status": "ok",
                         "ping_count": state.get("ping_count", 0)}}

HANDLERS = {"PING": handle_ping, "GET_STATUS": handle_get_status}

def router(state: State) -> State:
    command = state.get("pending_command")
    if command is None:
        return {"pending_command": None, "response": None}
    handler = HANDLERS.get(command.get("type"))
    if handler is None:
        raise ValueError("Unknown command type")
    update = handler(state)
    return {**update, "pending_command": None}       # the clear goes last

# {"type": "NOPE"} on the stream ->
#   error event {'error': 'ValueError', 'message': 'Unknown command type'}
```

STREAM_MODE="VALUES"

Keep the last snapshot

The first values event is the input still holding the command; the last is the answer

- Draining the stream is how a quiet PING still notices an error event.
- Sending response: None with each command drops a reply left by an earlier run.

PYTHON

```python
# PING on a fresh thread: two snapshots
# values {'pending_command': {'type': 'PING'}, 'response': None}   <- the input
# values {'pending_command': None, 'response': None, 'ping_count': 1}

def run_command(thread_id: str, command_type: str) -> dict:
    final_state = None
    for chunk in client.runs.stream(
        thread_id, "commands",
        input={"pending_command": {"type": command_type},
               "response": None},          # drop an older reply
        stream_mode="values",
    ):
        if chunk.event == "error":
            raise RuntimeError(chunk.data)
        if chunk.event == "values":
            final_state = chunk.data       # overwrite: the LAST one counts
    if final_state is None:
        raise RuntimeError("No state received")
    if final_state.get("pending_command") is not None:
        raise RuntimeError("Command was not cleared")
    return final_state
```

Two commands, one thread

Same thread id, same baton: GET_STATUS sees the count an earlier PING left

- PING prints only the thread id; the server log and the next command show it ran.
- Leave THREAD_ID unset and the client opens a new thread whose count starts at 0.

PYTHON

```python
# clients/common.py: reuse THREAD_ID when set, otherwise open a new thread
def get_thread_id() -> str:
    existing = os.getenv("THREAD_ID")
    if existing:
        return existing                              # same id, same baton
    return client.threads.create()["thread_id"]  # new id: the count starts at 0

# clients/get_status.py
thread_id = get_thread_id()
print(f"THREAD_ID={thread_id}")
state = run_command(thread_id, "GET_STATUS")
print(json.dumps(state.get("response"), indent=2))

# $ python clients/ping.py           -> THREAD_ID=01a16361-...   (no reply)
#   server log: PING handled; ping_count=1
# $ export THREAD_ID="01a16361-..."
# $ python clients/get_status.py     -> {"status": "ok", "ping_count": 1}
# second PING on that thread         -> "ping_count": 2
# THREAD_ID unset                    -> a new thread, "ping_count": 0
```

Directly or through a backend

Who checks the user and who owns the URLs differ; the graph is the same either way

- Option A: the browser calls the server's routes and keeps the thread id between clicks.
- Option B: your FastAPI checks the login, picks the thread, then forwards or runs the graph.

TEXT

```
Option A: the browser calls LangGraph Server directly
  [Ping] --POST /threads/{id}/runs/stream--> LangGraph Server --> router node
         assistant_id "commands"; the browser keeps the thread id
         login: the server's auth (langgraph.json "auth"), or nobody

Option B: the browser calls your FastAPI backend
  [Ping] --POST /api/ping--> FastAPI: check the user, pick the thread
                             |--> runs.stream to LangGraph Server
                             |      (the server still keeps the threads)
                             '--> graph.ainvoke(...) in this process
                                    (your checkpointer keeps the threads)

Same graph, same pending_command either way.
Products with logged-in users usually end up with B.
```

One builder, a second compile

Your own backend compiles the same builder WITH a checkpointer, once, at startup

- Before yield runs at startup, after it at shutdown, where async with closes the database.
- Without DB_URI it falls back to InMemorySaver: every thread is gone on a restart.

PYTHON

```python
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver
from server.graph import builder          # the same wiring the server loads

@asynccontextmanager
async def lifespan(app: FastAPI):
    db_uri = os.getenv("DB_URI")
    if db_uri:
        async with AsyncPostgresSaver.from_conn_string(db_uri) as saver:
            await saver.setup()           # creates the tables; safe to repeat
            app.state.graph = builder.compile(checkpointer=saver)
            logger.info("Checkpointer: PostgreSQL")
            yield
    else:
        app.state.graph = builder.compile(checkpointer=InMemorySaver())
        logger.warning("Checkpointer: memory. Set DB_URI to keep threads ...")
        yield

app = FastAPI(title="Pending commands", lifespan=lifespan)
app.include_router(api_router)
# On LangGraph Server: builder.compile()  (no checkpointer, the server owns it)
```

Turn a request into a user id

A dependency resolves the bearer token to a user before any endpoint body runs

- No header, or a token not in the table, gets a 401 before the graph runs.
- DEMO_TOKENS stands in for real login; the rest of the backend only needs a user id.

PYTHON

```python
DEMO_TOKENS = {"token-amira": "amira", "token-omar": "omar"}   # demo only
bearer = HTTPBearer()                # reads "Authorization: Bearer ..."

def current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer),
) -> str:
    user_id = DEMO_TOKENS.get(credentials.credentials)
    if user_id is None:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Unknown token")
    return user_id

@api_router.post("/ping", status_code=status.HTTP_204_NO_CONTENT)
async def ping(request: Request, user_id: str = Depends(current_user)) -> None:
    await run_command(request, user_id, "PING")      # runs after the check

# curl -i -X POST localhost:8000/api/ping -H "Authorization: Bearer token-amira"
#   HTTP/1.1 204 No Content
# curl localhost:8000/api/status -H "Authorization: Bearer token-omar"
#   {"status":"ok","ping_count":0}           <- Omar has his own thread
# curl localhost:8000/api/status             <- no header
#   {"detail":"Not authenticated"}           (401)
```

The backend picks the thread

The thread id comes from the user and the reply is a response model, not the state

- No URL takes a thread id, so a browser has nothing to tamper with.
- ainvoke returns only the final state, so the first-snapshot trap does not exist here.

PYTHON

```python
def thread_for(user_id: str) -> str:
    return f"commands-{user_id}"      # one thread per user, chosen on the server

async def run_command(request: Request, user_id: str, command_type: str) -> dict:
    graph = request.app.state.graph   # compiled once in the lifespan
    final_state = await graph.ainvoke(
        {"pending_command": {"type": command_type}, "response": None},
        config={"configurable": {"thread_id": thread_for(user_id)}},
    )
    if final_state.get("pending_command") is not None:
        raise RuntimeError("Command was not cleared")
    return final_state

class StatusOut(BaseModel):           # the browser's contract, not the state
    status: Literal["ok"]
    ping_count: int

@api_router.get("/status", response_model=StatusOut)
async def get_status(request: Request,
                     user_id: str = Depends(current_user)) -> StatusOut:
    state = await run_command(request, user_id, "GET_STATUS")
    return StatusOut(**state["response"])
```

From a laptop to a host

Production changes where the graph runs and who can reach it, not what it does

- Option A on a public page needs the server's auth handler, or anyone can run any thread.
- Option B: always set DB_URI, pool connections under load, and share threads only via Postgres.

BASH

```bash
# LangGraph Server (option A): the same CLI that ran langgraph dev
$ langgraph up                        # Docker: server + PostgreSQL + Redis, port 8123
$ langgraph up --postgres-uri "$DB_URI"     # use your own database
$ langgraph build -t commands-server        # an image for any container host
$ langgraph dockerfile Dockerfile           # or write a Dockerfile to edit
$ langgraph deploy                          # beta: LangSmith Deployment (hosted)
#   needs LANGSMITH_API_KEY locally, LANGGRAPH_CLOUD_LICENSE_KEY in production

# langgraph.json keys for real users:
#   "auth": {"path": "./auth.py:auth"}   an Auth handler checks every request
#   "http": {"app": "./webapp.py:app"}   mount your own FastAPI routes beside it
# A second run on a busy thread: multitask_strategy=
#   "reject" | "interrupt" | "rollback" | "enqueue"

# Your FastAPI service (option B)
$ DB_URI="postgresql://..." uvicorn backend.app:app --port 8000
```

Three quiet failures

A {} return, a raise before the clear, and a missing checkpointer all fail silently

- The slot clears only when the return includes pending_command set to None.
- A null slot is not an idempotency key: a side effect before a raise can repeat on retry.

PYTHON

```python
# 1. Returning {} to mean "done": the command stays in the slot
def router(state): return {}
# -> {'pending_command': {'type': 'PING'}}     the client: "Command was not cleared"

# 2. A handler that raises never reaches the clear
graph.invoke({"pending_command": {"type": "NOPE"}}, cfg)   # ValueError
graph.get_state(cfg).values
# -> {'pending_command': {'type': 'NOPE'}}     still set for a retry

# 3. Your own process, no checkpointer: thread_id is accepted, nothing saved
g0 = builder.compile()
cfg = {"configurable": {"thread_id": "x"}}
await g0.ainvoke({"pending_command": {"type": "PING"}}, config=cfg)   # twice
await g0.ainvoke({"pending_command": {"type": "GET_STATUS"}}, config=cfg)
# -> 'response': {'status': 'ok', 'ping_count': 0}   no error raised

# On LangGraph Server it is the reverse: compile with NO checkpointer.
# Checked in-process on langgraph 1.2.6 (the demo pins 1.2.16)
```